Lung Cancer Detection

Progress Report [02/Aug/2025]:

Project Objective:

Detect lung cancer from chest X-ray images using a deep learning model (EfficientNetV2), and provide predictions through an interactive Streamlit-based web interface.

1. 📁 Project Structure and Setup

- Created a modular folder structure:
  - models/: custom model code (e.g., EfficientNetV2)
  - utils/: preprocessing logic for dataset
  - app/: Streamlit web app
  - weights/: to store the trained model
- Setup Python virtual environment and installed dependencies via requirements.txt.

2. 🖼️ Dataset Handling

- Selected dataset: ("Lung and Colon Cancer Histopathological Images").
- Organized dataset into:

```
data/images/
├──── cancer/
└──── normal/
```

- Implemented preprocessing pipeline using torchvision:
  - Resizing images
  - Normalizing with transforms

- Splitting into train/validation loaders

3. 🧠 **Model Definition**

- Used a state-of-the-art deep learning architecture:
  - ✅ EfficientNetV2 (with pretrained ImageNet weights)
- Modified the classifier for binary classification (Cancer / Normal).

4. ⚙️ **Training Pipeline**

- Developed training script train_efficientnetv2.py:
  - Uses PyTorch with Adam optimizer
  - Tracks loss per epoch
  - Saves trained weights to weights/efficientnetv2.pth

5. 🌐 **Streamlit Web App**

- Built a simple and clean UI to upload X-ray images.
- Backend loads trained model and returns prediction instantly.
- Implemented image preprocessing and result rendering.

6. 🧪 **Current Output**

- Training has started successfully.
- Streamlit app launches correctly.
- Basic prediction pipeline is working with test images.

📸 Demo Preview (Optional)

If possible, run the Streamlit app and upload a sample image to show:

- The UI
- The prediction result

🛠️ **Upcoming Work (Next Milestones)**

1. Improve training (add validation accuracy and loss)
2. Add support for other models (ResNet, DenseNet)
3. Track metrics and save training logs
4. Improve UI with probability score, class explanation, etc.
5. Package app for deployment (e.g., Hugging Face Spaces or Streamlit Cloud)